010_Dynamic

1. 核心定义 | Core Definition

动态规划（简称 **DP**）是一种解决复杂问题的方法：它通过将大问题分解为**相互重叠的子问题**（Overlapping sub-problems），并存储子问题的解来避免重复计算。

- **贪婪算法**：只看当前，不走回头路（局部最优）。
- **动态规划**：统筹全局，通过一张**网格图 (Grid)** 记录所有可能性，最终找到绝对的最优解。

---

2. 经典案例：背包问题 | The Knapsack Problem

还记得上一章打饭的例子吗？动态规划能帮你算出在空间有限的情况下，如何拿取组合能让总价值达到最大。

假设你的背包容量是 **4kg**，商店里有三件商品：

1. **音响**：$3000，4kg
2. **笔记本电脑**：$2000，3kg
3. **吉他**：$1500，1kg

贪婪算法会先拿音响（最贵），然后背包满了。总价：$3000。

动态规划会通过网格计算发现：拿电脑 + 吉他。总价：$3500。

---

3. 网格是 DP 的灵魂 | The Grid is the Soul of DP

在 DP 中，你会画一张表。每一行代表**可选的商品**，每一列代表**当前背包的容量**。

商品	1kg	2kg	3kg	4kg
吉他	$1500	$1500	$1500	$1500
电脑	$1500	$1500	$2000	**$3500** (吉他+电脑)
音响	$1500	$1500	$2000	$3500 (保持不变)

计算公式 (The Formula)：

每一个格子的价值 = $\max$ (上一个单元格的价值, 当前商品的价值 + 剩余空间的价值)。

```python
# 物品信息：(价值, 重量)
items = {
    "guitar": (1500, 1),
    "laptop": (2000, 3),
    "stereo": (3000, 4)
}

# 背包容量
capacity = 4
item_list = list(items.keys())

# 初始化网格 (Rows: 物品, Cols: 容量 1kg~4kg)
# cell[i][j] 代表在前 i 个物品中，容量为 j 时能装的最大价值
grid = [[0] * (capacity + 1) for _ in range(len(item_list))]

for i in range(len(item_list)):
    name = item_list[i]
    value, weight = items[name]

    for j in range(1, capacity + 1):
        # 当前背包容量 j
        if i == 0: # 第一行（只有吉他）
            grid[i][j] = value if weight <= j else 0
        else:
            prev_max = grid[i-1][j] # 上一个格子的最优解

            if weight <= j:
                # 核心公式：max(不装当前物品, 装当前物品 + 剩余空间的最优解)
                # 剩余空间价值 = grid[i-1][j-weight]
                grid[i][j] = max(prev_max, value + grid[i-1][j-weight])
            else:
                grid[i][j] = prev_max

print(f"最大价值为: {grid[-1][-1]}") # 输出 3500
```

4. 动态规划的局限性 | Limitations of DP

- **不可分性**：DP 只适用于“离散”的物体（拿或不拿）。如果你可以把商品切开（比如散装面粉），那用**贪婪算法**更好。
- **相互依赖**：DP 假设每个子问题是相互独立的。如果商品 A 和商品 B 必须成对出现，网格逻辑就需要变得更复杂。

5. 什么时候该用 DP？ | When to Use DP?

如果你遇到以下关键词，大脑里要立刻蹦出“动态规划”：

1. **在给定约束条件下追求最优解**（Optimization under constraints）。
2. **问题可以分解为彼此依赖的离散子问题**。
3. **计算最长公共子序列（LCS）**：例如对比两段 DNA 的相似度，或者拼写检查。